

PANDUAN KONFIGURASI DOCKER

VM1 Deployment dan VM2 Management

Laravel + Apache + MariaDB + Nginx Load Balancer
3 Web Container pada Setiap VM

VM1	berke-deployment Aplikasi Laravel Port 8080 dan 3306
VM2	berke-management 3 Nginx web Port 8080
Network Docker	network-ujikom
Volume VM1	volume-ujikom

*Dokumen ini hanya membahas Docker dan deployment aplikasi.
Konfigurasi jaringan, SSH, dan FTP tidak disertakan.*

Versi final - konfigurasi yang berhasil dijalankan

1. Cakupan dan Arsitektur

Panduan ini merangkum langkah Docker yang sudah berhasil digunakan pada dua VM. Semua file konfigurasi dibuat menggunakan perintah cat dengan here-document agar mudah disalin saat praktik.

Tidak dibahas: Netplan, IP statis, SSH passwordless, firewall FTP, dan instalasi vsftpd.

Arsitektur VM1

- 1 container MariaDB 11.4 dengan volume persisten volume-ujikom.
- 3 container aplikasi Laravel berbasis custom image PHP 8.3 Apache.
- 1 container Nginx sebagai load balancer pada port host 8080.
- Web container hanya expose port 80 dan tidak publish port langsung ke host.
- Database publish port 3306 ke host.

Arsitektur VM2

- 3 container Nginx sebagai web server sederhana.
- 1 container Nginx sebagai load balancer pada port host 8080.
- Web container hanya expose port 80.
- Semua service menggunakan restart policy unless-stopped.

Ringkasan direktori

Bagian	Direktori	File utama
VM1	/home/ujikom/hotel-online	Dockerfile, entryptpoint.sh, .env.docker, nginx.conf, docker-compose.yml
VM2	/home/ujikom/vm2-loadbalancer	start-web.sh, nginx.conf, docker-compose.yml

2. VM1 Deployment - Laravel, MariaDB, dan Load Balancer

Alamat latihan: Contoh APP_URL menggunakan http://192.168.10.220:8080. Ganti nilai tersebut jika IP VM1 berubah.

2.1. Instal Docker dan alat pendukung

```
sudo apt update
sudo apt install -y \
  ca-certificates \
  curl \
  git \
  gnupg \
  openssl \
  nano \
  unzip \
  jq \
  tree \
  docker.io \
  docker-compose-v2

sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

Logout lalu login kembali agar grup docker aktif, kemudian verifikasi:

```
docker --version
docker compose version
docker run --rm hello-world
```

2.2. Siapkan direktori dan clone project

```
sudo mkdir -p /home/ujikom
sudo chown -R student:student /home/ujikom
cd /home/ujikom

git clone https://github.com/syberke/hotel-online.git
cd hotel-online
git switch main
git pull origin main
```

```
pwd
ls -la
```

2.3. Buat .dockerignore

```
cat > .dockerignore <<'EOF'
.git
.env
.env.docker
node_modules
vendor
public/build
public/hot
storage/logs/*
storage/framework/cache/*
storage/framework/sessions/*
storage/framework/views/*
oasis_hotel_backup.sql
EOF
```

2.4. Buat entrypoint.sh

```
cat > entrypoint.sh <<'EOF'
#!/bin/sh
set -e

cd /var/www/html
rm -f public/hot

if [ ! -f .env ] && [ -f .env.example ]; then
    cp .env.example .env
fi

mkdir -p \
    storage/framework/cache/data \
    storage/framework/sessions \
    storage/framework/views \
    storage/logs \
    bootstrap/cache

chown -R www-data:www-data storage bootstrap/cache
chmod -R ug+rwX storage bootstrap/cache

if [ "${DB_CONNECTION:-mysql}" = "mysql" ]; then
    echo "Menunggu MariaDB..."

    until mysqladmin ping \
        -h "${DB_HOST:-database}" \
        -P "${DB_PORT:-3306}" \
        -u "${DB_USERNAME}" \
        -p"${DB_PASSWORD}" \
        --silent; do
        sleep 3
    done
fi

php artisan optimize:clear
php artisan storage:link --force || true

if [ "${RUN_MIGRATIONS:-false}" = "true" ]; then
    php artisan migrate --force
fi

exec "$@"
EOF

chmod +x entrypoint.sh
```

2.5. Buat Dockerfile final

```
cat > Dockerfile <<'EOF'
FROM php:8.3-apache

ENV APACHE_DOCUMENT_ROOT=/var/www/html/public

RUN apt-get update && apt-get install -y \
    git \
    unzip \
    curl \
    default-mysql-client \
    nodejs \
    npm \
    libonig-dev \
    libzip-dev \
    libicu-dev \
    libpng-dev \
```

```

libjpeg62-turbo-dev \
libfreetype6-dev \
&& docker-php-ext-configure gd --with-jpeg --with-freetype \
&& docker-php-ext-install \
    pdo_mysql \
    mbstring \
    zip \
    intl \
    gd \
    bcmath \
&& a2enmod rewrite headers \
&& sed -ri 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' \
    /etc/apache2/sites-available/*.conf \
    /etc/apache2/apache2.conf \
    /etc/apache2/conf-available/*.conf \
&& rm -rf /var/lib/apt/lists/*

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html
COPY . .

RUN composer install \
    --no-interaction \
    --prefer-dist \
    --optimize-autoloader \
&& npm config set registry https://registry.npmjs.org/ \
&& npm config set fetch-retries 5 \
&& npm config set fetch-retry-factor 2 \
&& npm config set fetch-retry-mintimeout 20000 \
&& npm config set fetch-retry-maxtimeout 120000 \
&& npm install --no-audit --no-fund \
&& npm run build \
&& chown -R www-data:www-data storage bootstrap/cache

COPY entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

EXPOSE 80

ENTRYPOINT ["entrypoint.sh"]
CMD ["apache2-foreground"]
EOF

```

Penting: Pada deployment Docker tidak perlu menjalankan npm run dev atau php artisan serve. Asset dibuat dengan npm run build dan aplikasi dilayani Apache.

2.6. Buat .env.docker

```

APP_KEY=$(printf 'base64:%s' "$(openssl rand -base64 32)")

cat > .env.docker <<EOF
APP_NAME="Oasis Hotel"
APP_ENV=production
APP_KEY=${APP_KEY}
APP_DEBUG=false
APP_URL=http://192.168.10.220:8080

LOG_CHANNEL=stack
LOG_LEVEL=error

DB_CONNECTION=mysql
DB_HOST=database
DB_PORT=3306
DB_DATABASE=oasis_hotel
DB_USERNAME=oasis_hotel
DB_PASSWORD=UkkHotel123!

MARIADB_DATABASE=oasis_hotel
MARIADB_USER=oasis_hotel
MARIADB_PASSWORD=UkkHotel123!
MARIADB_ROOT_PASSWORD=RootUkk123!

SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync

RUN_MIGRATIONS=false

MIDTRANS_SERVER_KEY=
MIDTRANS_CLIENT_KEY=
MIDTRANS_IS_PRODUCTION=false

RECAPTCHA_SITE_KEY=

```

```
RECAPTCHA_SECRET_KEY=

MAIL_MAILER=log
EOF

chmod 600 .env.docker
```

2.7. Buat nginx.conf load balancer

```
cat > nginx.conf <<'EOF'
resolver 127.0.0.11 valid=5s ipv6=off;

upstream hotel_web {
    zone hotel_web 64k;
    server web:80 resolve;
}

server {
    listen 80;
    server_name _;

    location = /lb-health {
        default_type text/plain;
        return 200 "load balancer aktif\n";
    }

    location / {
        proxy_pass http://hotel_web;
        proxy_http_version 1.1;

        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-Host $http_host;
        proxy_set_header X-Forwarded-Port 8080;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 10s;
        proxy_read_timeout 60s;

        add_header X-Load-Balancer "nginx-ukk" always;
        add_header X-Upstream-Address $upstream_addr always;
    }
}
EOF
```

Kenapa \$http_host: Variabel ini mempertahankan host beserta port 8080 agar URL asset Laravel tidak kehilangan nomor port.

2.8. Buat docker-compose.yml

```
cat > docker-compose.yml <<'EOF'
services:
  web:
    build: .
    restart: unless-stopped
    env_file:
      - .env.docker
    expose:
      - "80"
    depends_on:
      database:
        condition: service_healthy
    networks:
      - network-ujikom

  database:
    image: mariadb:11.4
    restart: unless-stopped
    env_file:
      - .env.docker
    ports:
      - "3306:3306"
    volumes:
      - volume-ujikom:/var/lib/mysql
    healthcheck:
      test:
        - CMD-SHELL
        - healthcheck.sh --connect --innodb_initialized
      interval: 5s
      timeout: 3s
      retries: 20
    networks:
      - network-ujikom
```

```
loadbalancer:
  image: nginx:1.27-alpine
  restart: unless-stopped
  ports:
    - "8080:80"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    - web
  networks:
    - network-ujikom

volumes:
  volume-ujikom:
    name: volume-ujikom

networks:
  network-ujikom:
    name: network-ujikom
    driver: bridge
EOF
```

2.9. Tambahkan route bukti load balancing

Backup dahulu lalu tambahkan route /instance ke bagian akhir routes/web.php:

```
cp routes/web.php routes/web.php.backup
cat >> routes/web.php <<'EOF'

Route::get('/instance', function () {
    $hostname = gethostname() ?: 'unknown-container';

    $html = '<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bukti Load Balancer</title>
  <style>
    body{margin:0;min-height:100vh;display:grid;place-items:center;
      background:#0f172a;color:#e2e8f0;font-family:Arial,sans-serif}
    main{width:min(560px,calc(100% - 40px));padding:36px;
      border:1px solid #334155;border-radius:24px;background:#111827}
    .status{display:inline-block;padding:8px 12px;border-radius:999px;
      background:#064e3b;color:#a7f3d0;font-weight:700}
    .host{margin-top:20px;padding:18px;border-radius:14px;background:#1e293b;
      font-family:monospace;font-size:22px;color:#93c5fd;word-break:break-all}
    p{line-height:1.7;color:#94a3b8}
  </style>
</head>
<body>
  <main>
    <span class="status">LOAD BALANCER AKTIF</span>
    <h1>Oasis Hotel Online</h1>
    <p>Refresh halaman ini untuk melihat hostname container.</p>
    <div class="host">Container: '.e($hostname).'

```

Catatan: Pastikan routes/web.php sudah memiliki use Illuminate\Support\Facades\Route; pada bagian atas file.

2.10. Validasi dan build image

```
docker compose --env-file .env.docker config
docker compose --env-file .env.docker build web
```

Jika npm ECONNRESET: Jalankan ulang perintah build yang sama. Layer sebelumnya tersimpan dalam cache sehingga proses berikutnya biasanya jauh lebih cepat.

2.11. Jalankan database, migration, dan seeder

```
docker compose --env-file .env.docker up -d database
sleep 15
docker compose --env-file .env.docker ps
```

```
docker compose --env-file .env.docker run --rm web \
  php artisan migrate --force

docker compose --env-file .env.docker run --rm web \
  php artisan db:seed --force

docker compose --env-file .env.docker run --rm web \
  php artisan migrate:status
```

Opsional: buat akun admin

```
docker compose --env-file .env.docker run --rm web php artisan tinker --execute="
\App\Models\User::updateOrCreate(
  ['email' => 'admin@oasis.test'],
  [
    'name' => 'Admin UKK',
    'password' => \Illuminate\Support\Facades\Hash::make('Admin123!'),
    'role' => 'admin',
    'account_status' => 'active',
    'email_verified_at' => now(),
  ]
);
"
```

Akun admin: Email admin@oasis.test dan password Admin123!. Jalankan hanya bila tabel users memiliki kolom role dan account_status.

2.12. Jalankan 3 web container dan load balancer

```
docker compose --env-file .env.docker up -d \
  --force-recreate \
  --scale web=3

docker compose --env-file .env.docker ps
```

2.13. Verifikasi VM1

```
docker ps --format 'table {{.Names}} \t {{.Status}} \t {{.Ports}}'

curl http://localhost:8080/lb-health

for i in {1..10}; do
  curl -s http://localhost:8080/instance \
    | grep -o 'Container: [^<]*'
done

docker network inspect network-ujikom >/dev/null && echo "network OK"
docker volume inspect volume-ujikom >/dev/null && echo "volume OK"
```

Hasil yang benar:

- Database berstatus healthy dan membuka 3306:3306.
- Tiga container web aktif dan hanya menampilkan 80/tcp.
- Load balancer membuka 8080:80.
- Route /instance bergantian menampilkan tiga hostname container.
- network OK dan volume OK.

2.14. Verifikasi asset CSS dan JavaScript

```
curl -s -H 'Host: 192.168.10.220:8080' http://localhost:8080 \
  | grep -Eo '(href|src)="[^"]+"' \
  | grep build

CSS=$(curl -s http://localhost:8080 \
  | grep -oE '/build/assets/[^\"]+\.css' \
  | head -n 1)

echo "$CSS"
curl -I "http://localhost:8080${CSS}"
```

Target: Asset mengarah ke /build/assets/... dan memberi HTTP 200 dengan Content-Type text/css.

3. VM2 Management - 3 Nginx Web dan Load Balancer

3.1. Instal Docker

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

Logout dan login kembali, lalu verifikasi:

```
docker --version
docker compose version
docker run --rm hello-world
```

3.2. Buat direktori project

```
sudo mkdir -p /home/ujikom/vm2-loadbalancer
sudo chown -R student:student /home/ujikom
cd /home/ujikom/vm2-loadbalancer
```

3.3. Buat start-web.sh

```
cat > start-web.sh <<'EOF'
#!/bin/sh
set -e

HOST="$(hostname)"

cat > /usr/share/nginx/html/index.html <<HTML
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>VM2 Management</title>
  <style>
    body {
      min-height: 100vh;
      margin: 0;
      display: grid;
      place-items: center;
      background: #0f172a;
      color: #e2e8f0;
      font-family: Arial, sans-serif;
    }
    main {
      padding: 40px;
      border: 1px solid #334155;
      border-radius: 24px;
      background: #111827;
      text-align: center;
    }
    strong {
      display: block;
      margin-top: 20px;
      color: #93c5fd;
      font-size: 24px;
    }
  </style>
</head>
<body>
  <main>
    <h1>VM2 Management</h1>
    <p>Load balancing aktif</p>
    <strong>Container: ${HOST}</strong>
  </main>
</body>
</html>
HTML

exec nginx -g "daemon off;";
EOF

chmod +x start-web.sh
```

3.4. Buat nginx.conf

```
cat > nginx.conf <<'EOF'
resolver 127.0.0.11 valid=5s ipv6=off;

upstream vm2_web {
  zone vm2_web 64k;
```



```

server web:80 resolve;
}

server {
    listen 80;
    server_name _;

    location = /lb-health {
        default_type text/plain;
        return 200 "VM2 load balancer aktif\n";
    }

    location / {
        proxy_pass http://vm2_web;
        proxy_http_version 1.1;

        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        add_header X-Load-Balancer "vm2-nginx" always;
        add_header X-Upstream-Address $upstream_addr always;
    }
}
EOF

```

3.5. Buat docker-compose.yml

```

cat > docker-compose.yml <<'EOF'
services:
  web:
    image: nginx:1.27-alpine
    restart: unless-stopped
    expose:
      - "80"
    volumes:
      - ./start-web.sh:/usr/local/bin/start-web.sh:ro
    command:
      - /bin/sh
      - /usr/local/bin/start-web.sh
    networks:
      - network-ujikom

  loadbalancer:
    image: nginx:1.27-alpine
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - web
    networks:
      - network-ujikom

networks:
  network-ujikom:
    name: network-ujikom
    driver: bridge
EOF

```

3.6. Validasi dan jalankan 3 container web

```

docker compose config
docker compose up -d --scale web=3

docker compose ps
docker ps --format 'table {{.Names}} \t {{.Status}} \t {{.Ports}}'

```

3.7. Verifikasi VM2

```

curl http://localhost:8080/lb-health

for i in {1..10}; do
  curl -s http://localhost:8080 \
  | grep -o 'Container: [^<]*'
  sleep 1
done

```

Target: Muncul 3 hostname container secara bergantian. Load balancer membuka 8080:80 dan ketiga web hanya expose 80/tcp.

4. Operasional, Perubahan IP, dan Troubleshooting

4.1 Perubahan IP VM1

Docker tidak perlu diinstal ulang dan image tidak perlu dibangun ulang. Ubah APP_URL lalu recreate container:

```
cd /home/ujikom/hotel-online
nano .env.docker

# Ubah menjadi IP baru, contoh:
APP_URL=http://172.20.3.63:8080

docker compose --env-file .env.docker up -d \
  --force-recreate \
  --scale web=3

for container in $(docker compose --env-file .env.docker ps -q web); do
  docker exec "$container" php artisan optimize:clear
done
```

4.2 Reboot dan restart otomatis

Karena seluruh service memakai restart: unless-stopped, container akan aktif kembali setelah Docker menyala.

```
sudo reboot

# Setelah VM aktif kembali:
cd /home/ujikom/hotel-online
docker compose --env-file .env.docker ps
curl http://localhost:8080/lb-health
```

Untuk VM2:

```
cd /home/ujikom/vm2-loadbalancer
docker compose ps
curl http://localhost:8080/lb-health
```

4.3 Perintah administrasi yang berguna

```
# Lihat semua container
docker ps

# Lihat log VM1
docker compose --env-file .env.docker logs --tail=100 web
docker compose --env-file .env.docker logs --tail=100 database
docker compose --env-file .env.docker logs --tail=100 loadbalancer

# Restart load balancer VM1
docker compose --env-file .env.docker restart loadbalancer

# Recreate VM1 tanpa build ulang
docker compose --env-file .env.docker up -d --force-recreate --scale web=3

# Stop tanpa menghapus volume
docker compose --env-file .env.docker down

# Jalankan kembali
docker compose --env-file .env.docker up -d --scale web=3
```

4.4 Larangan penting

```
docker compose down -v
```

Jangan jalankan sembarangan: Opsi -v menghapus named volume. Pada VM1, data MariaDB di volume-ujikom dapat hilang.

4.5 Troubleshooting cepat

Masalah	Penyebab umum	Solusi
npm ERR! ECONNRESET	Koneksi registry npm terputus saat build	Jalankan ulang docker compose build web. Cache membuat pengulangan lebih cepat.
CSS tidak tampil	public/hot masih ada atau URL asset kehilangan port	Hapus public/hot, gunakan npm run build, dan proxy_set_header Host \$http_host.

Masalah	Penyebab umum	Solusi
Database belum healthy	MariaDB masih inisialisasi atau env salah	Tunggu 15-30 detik dan lihat logs database.
Hanya satu web muncul	Service dijalankan tanpa --scale web=3	Jalankan <code>up -d --force-recreate --scale web=3</code> .
Perubahan .env tidak terbaca	docker compose restart tidak membuat container baru	Gunakan <code>up -d --force-recreate</code> .
Port 8080 gagal dibuka	Port dipakai proses lain	Periksa <code>sudo ss -ltnp grep :8080</code> lalu hentikan service yang bentrok.

5. Checklist Akhir

VM1

- Custom image berhasil dibangun.
- MariaDB healthy dan data tersimpan di volume-ujikom.
- Tiga web container aktif.
- Load balancer aktif pada port 8080.
- Database publish port 3306.
- network-ujikom dan volume-ujikom tersedia.
- Route /instance berpindah di antara tiga hostname.
- CSS dan JavaScript dari public/build memberi HTTP 200.

VM2

- Tiga Nginx web container aktif.
- Satu Nginx load balancer aktif pada port 8080.
- Web container hanya expose port 80.
- Halaman bergantian menampilkan tiga hostname container.
- Container kembali aktif setelah reboot.

Selesai: Jika seluruh checklist terpenuhi, konfigurasi Docker VM1 dan VM2 sudah siap digunakan untuk demonstrasi dan penilaian.